

School of Computer Science and Artificial Intelligence

Lab Assignment # 11.3

Program	: B. Tech (CSE)
Specialization	:
Course Title	: AI Assisted coding
Course Code	:
Semester	: II
Academic Session	: 2025-2026
Name of Student	: M.Srinath
Enrollment No.	: 2403A51L29
Batch No.	: 51
Date	:24-02-2026

Task 1: Smart Contact Manager (Arrays & Linked Lists)**AI Prompt Used:**

Generate Python code for a Contact Manager using arrays and linked lists with add, search, and delete operations.

Array Implementation Explanation:

In this implementation, Python lists are used as arrays to store contact dictionaries. Insertion is efficient at the end ($O(1)$), but search and deletion require traversal ($O(n)$).

Array Implementation Code:

```
task1.py > ContactManagerLinkedList > delete_contact
1 class ContactManagerArray:
2     def __init__(self):
3         self.contacts = []
4
5     def add_contact(self, name, phone):
6         self.contacts.append({"name": name, "phone": phone})
7         print("Contact Added Successfully")
8
9     def search_contact(self, name):
10        for contact in self.contacts:
11            if contact["name"] == name:
12                return contact
13        return "Contact Not Found"
14
15    def delete_contact(self, name):
16        for contact in self.contacts:
17            if contact["name"] == name:
18                self.contacts.remove(contact)
19                return "Contact Deleted"
20        return "Contact Not Found"
21
22
23
24 cm = ContactManagerArray()
25 cm.add_contact("Rahul", "9876543210")
26 cm.add_contact("Anjali", "9123456789")
27
28 print(cm.search_contact("Rahul"))
29 print(cm.delete_contact("Rahul"))
30 print(cm.search_contact("Rahul"))
```

Linked List Explanation:

Linked List uses dynamic memory allocation. Each node stores contact details and a reference to the next node. Insertion at the beginning is $O(1)$. Search and delete require traversal $O(n)$.

Linked List Code:

```
39 class ContactManagerLinkedList:
40     def __init__(self):
41         self.head = None
42
43     def add_contact(self, name, phone):
44         new_node = Node(name, phone)
45         new_node.next = self.head
46         self.head = new_node
47         print("Contact Added Successfully")
48
49     def search_contact(self, name):
50         temp = self.head
51         while temp:
52             if temp.name == name:
53                 return {"name": temp.name, "phone": temp.phone}
54             temp = temp.next
55         return "Contact Not Found"
56
57     def delete_contact(self, name):
58         temp = self.head
59         prev = None
60
61         while temp:
62             if temp.name == name:
63                 if prev:
64                     prev.next = temp.next
65                 else:
66                     self.head = temp.next
67                 return "Contact Deleted"
```

Sample Output:

```
PS D:\python> python task1.py
Contact Added Successfully
Contact Added Successfully
{'name': 'Rahul', 'phone': '9876543210'}
Contact Deleted
Contact Not Found
PS D:\python>
```

Task 2: Library Book Search System (Queue & Priority Queue)

AI Prompt Used:

Generate enqueue and dequeue methods for Queue and Priority Queue in Python.

Queue Explanation:

Queue follows FIFO (First In First Out). Requests are handled in the order received.

Queue Code:

```
task2.py / ...
1  from collections import deque
2
3  class LibraryQueue:
4      def __init__(self):
5          self.queue = deque()
6
7      def enqueue(self, request):
8          self.queue.append(request)
9
10     def dequeue(self):
11         if self.queue:
12             return self.queue.popleft()
13         return "No Requests"
14
15
16
17 lq = LibraryQueue()
18 lq.enqueue("Student - Book A")
19 lq.enqueue("Student - Book B")
20
21 print(lq.dequeue())
22 print(lq.dequeue())
```

Priority Queue Explanation:

Priority Queue assigns higher priority to faculty requests using heapq. Faculty requests are processed before student requests

Sample Output:

```
PS D:\python> python task2.py
Student - Book A
Student - Book B
PS D:\python> 
```

Task 3: Emergency Help Desk (Stack)

AI Prompt Used:

Generate stack operations push, pop, peek, is_empty in Python.

Explanation:

Stack follows LIFO (Last In First Out). The most recent ticket is resolved first.

Stack Code:

```
task3.py > ...
1  class HelpDeskStack:
2      def __init__(self):
3          self.stack = []
4          self.max_size = 10
5
6      def push(self, ticket):
7          if len(self.stack) < self.max_size:
8              self.stack.append(ticket)
9          else:
10             print("Stack Overflow")
11
12     def pop(self):
13         if self.stack:
14             return self.stack.pop()
15         return "Stack Underflow"
16
17     def peek(self):
18         if self.stack:
19             return self.stack[-1]
20         return "Stack Empty"
21
22     def is_empty(self):
23         return len(self.stack) == 0
24
25     def is_full(self):
26         return len(self.stack) == self.max_size
27
28
```

Sample Output:

```
PS D:\python> python task3.py
Email Not Working
Printer Error
PS D:\python>
```

Task 4: Hash Table with Chaining

AI Prompt Used:

Generate a HashTable class in Python with insert, search, delete and collision handling using chaining.

Explanation:

Hash Table uses a hash function to map keys to indices. Collisions are handled using chaining (list of key-value pairs at each index).

Hash Table Code:

```
task4.py > HashTable > delete
1  class HashTable:
2      def __init__(self, size=10):
3          self.size = size
4          self.table = [[] for _ in range(size)]
5
6      def hash_function(self, key):
7          return hash(key) % self.size
8
9      def insert(self, key, value):
10         index = self.hash_function(key)
11         for pair in self.table[index]:
12             if pair[0] == key:
13                 pair[1] = value
14                 return
15         self.table[index].append([key, value])
16
17     def search(self, key):
18         index = self.hash_function(key)
19         for pair in self.table[index]:
20             if pair[0] == key:
21                 return pair[1]
22         return "Key Not Found"
23
24     def delete(self, key):
25         index = self.hash_function(key)
26         for i, pair in enumerate(self.table[index]):
27             if pair[0] == key:
28                 self.table[index].pop(i)
29                 return "Key Deleted"
30         return "Key Not Found"
```

Sample Output:

```
PS D:\python> python task4.py
Inserted key 'apple' at index 1
Inserted key 'banana' at index 1
Inserted key 'melon' at index 3
Inserted key 'grape' at index 1
[[[]], [['apple', 100], ['banana', 200], ['grape', 500]], [], [['melon', 400]], [['orange', 300]]]
Search for banana: 200
Search for cherry: Key Not Found
Deleted key 'orange' from index 4
Key Deleted
Key 'cherry' not found to delete
Key Not Found
[[[]], [['apple', 100], ['banana', 200], ['grape', 500]], [], [['melon', 400]], []]
PS D:\python>
```

Task 5: Real-Time Application Challenge

Mapping Table:

Attendance Tracking → Hash Table (Fast search)
 Event Registration → Linked List (Dynamic size)
 Library Borrowing → Queue (FIFO)
 Bus Scheduling → Graph (Routes mapping)
 Cafeteria Orders → Queue (Sequential processing)

Selected Feature Implementation (Attendance System):

```
task5.py > ...
1 class AttendanceSystem:
2     def __init__(self):
3         self.records = {}
4
5     def mark_attendance(self, student_id, status):
6         self.records[student_id] = status
7
8     def check_attendance(self, student_id):
9         return self.records.get(student_id, "No Record Found")
10
11
12 attendance = AttendanceSystem()
13 attendance.mark_attendance("23CS101", "Present")
14 attendance.mark_attendance("23CS102", "Absent")
15
16 print(attendance.check_attendance("23CS101"))
```

Sample Output

```
PS D:\python> python task5.py  
Present  
PS D:\python> 
```